

INVENTORYHUB – SECURE INVENTORY MANAGEMENT PLATFORM

FINAL LAB PROJECT REPORT

Submitted By:

Anees Ahmed Paracha

SP23-BCT-006

M. Umair Sohail

SP23-BCT-039

Shariq Ali Jia Abbasi

SP23-BCT-047

Waleed Ahmed Farooqi

SP23-BCT-051

Submitted To:

Sir Muhammad Ahmad

Table Of Contents

Abstract	4
Chapter 1: Introduction	4
1.1 Background	4
1.2 Project Objective	4
1.3 Project Scope	5
Chapter 2: Security Requirements and Threat Modelling	5
2.1 Security Requirements	5
2.2 User Roles	6
2.3 Threat Modelling	6
2.4 Use Case Analysis	6
2.5 Data Flow Analysis	6
2.6 STRIDE Analysis	7
Chapter 3: Secure Architecture Design	7
3.1 Architecture Overview	7
3.2 Authentication and Authorization	8
3.3 Supporting Services	8
3.4 Zero Trust Security	8
Chapter 4: Secure Implementation and Testing	8
4.1 Security Controls Implemented	8
4.2 Static Application Security Testing	9
4.3 Dynamic Security Testing	10
4.4 Penetration Testing	10
4.5 Security Testing Results	10
Chapter 5: Container Security and Infrastructure Automation	10
5.1 Docker Security	10
5.2 Kubernetes Security	11
5.3 Infrastructure as Code	11
5.4 Secrets Management	11
Chapter 6: Monitoring, DevSecOps, and AI Security	11
6.1 DevSecOps Pipeline	11

6.2 Security Monitoring	12
6.3 AI-Based Anomaly Detection	12
Chapter 7: Results and Conclusion	12
7.1 Results	12
7.2 Conclusion	12
References	13

Abstract

InventoryHub is a secure inventory management platform developed following secure software development principles. The project was completed through multiple phases including security requirements engineering, threat modelling, secure architecture design, secure implementation, security testing, infrastructure automation, monitoring, and DevSecOps integration.

The system provides secure inventory management functionality while protecting critical business assets such as inventory records, supplier information, purchase orders, user accounts, and audit logs. Security controls including authentication, role-based access control, input validation, audit logging, security monitoring, container hardening, and infrastructure security were implemented. Security validation was performed through static analysis, dynamic testing, penetration testing, and compliance verification. The project demonstrates a practical implementation of secure software development practices across the complete software lifecycle.

Chapter 1: Introduction

1.1 Background

Modern inventory management systems handle critical business information including stock records, supplier information, purchase orders, and user accounts. Security weaknesses in such systems may lead to unauthorized access, inventory manipulation, financial losses, and operational disruptions. Therefore, security must be integrated throughout the software development lifecycle.

1.2 Project Objective

The objective of InventoryHub is to design and implement a secure inventory management platform capable of protecting inventory operations against common cyber threats while maintaining confidentiality, integrity, availability, and accountability.

The project objectives include:

- Secure inventory management operations
- Role-based access control

- Authentication and authorization
- Security monitoring and audit logging
- Threat modelling and risk assessment
- Secure deployment using containers
- Infrastructure automation and secrets management
- Continuous security validation

1.3 Project Scope

The system supports inventory management, supplier management, stock updates, reporting, audit logging, security monitoring, and secure deployment. The project covers security requirements engineering, architecture design, implementation, testing, infrastructure automation, monitoring, and DevSecOps practices.

Chapter 2: Security Requirements and Threat Modelling

2.1 Security Requirements

Security requirements were identified during the initial phase of development. The requirements focused on authentication, authorization, inventory protection, supplier data protection, audit logging, encryption, input validation, monitoring, and recovery mechanisms.

Critical assets identified include:

- User accounts and credentials
- Inventory records
- Supplier information
- Purchase and order records
- Audit logs
- Database contents
- Authentication tokens

2.2 User Roles

The platform defines four major roles:

Admin

Manages users, permissions, audit logs, and security settings.

Inventory Manager

Manages inventory items, suppliers, stock updates, and reports.

Staff/User

Performs authorized inventory operations.

Auditor

Reviews reports, audit logs, and security events.

2.3 Threat Modelling

Threat modelling was performed using Microsoft Threat Modelling Tool.

The threat model identified trust boundaries, data flows, external entities, and security controls. Potential attack paths were analyzed before implementation.

2.4 Use Case Analysis

The use case diagram illustrates system interactions between Admin, Inventory Manager, Staff, and Auditor while enforcing authentication, authorization, and audit logging.

2.5 Data Flow Analysis

The DFD illustrates secure movement of inventory information, user requests, supplier records, audit logs, and security monitoring events.

2.6 STRIDE Analysis

The STRIDE framework was used to identify threats.

Threat	Description	Mitigation
Spoofing	Unauthorized identity impersonation	Authentication and RBAC (Role-Based Access Control)
Tampering	Data modification	Input validation and access control
Repudiation	Denial of actions	Audit logging
Information Disclosure	Sensitive data exposure	Authorization and encryption
Denial of Service (DoS)	Service disruption	Rate limiting
Elevation of Privilege	Unauthorized privilege gain	RBAC enforcement

Chapter 3: Secure Architecture Design

3.1 Architecture Overview

The platform follows a layered secure architecture consisting of:

- User Access Layer
- Application Layer
- Database Layer
- Supporting Services Layer
- Monitoring Layer

3.2 Authentication and Authorization

Authentication is performed using JWT tokens. Authorization is enforced using Role-Based Access Control (RBAC) where each role receives only the permissions required for its responsibilities.

3.3 Supporting Services

MongoDB

Stores inventory data, users, suppliers, orders, and audit logs.

Redis

Provides temporary caching and failed-login tracking.

Kafka

Handles event-driven communication and security event streaming.

3.4 Zero Trust Security

The architecture follows Zero Trust principles where no user, request, service, or component is trusted by default. Every access request is authenticated, authorized, and validated before processing.

Chapter 4: Secure Implementation and Testing

4.1 Security Controls Implemented

The following controls were implemented:

HTTP Security Headers

Helmet middleware was used to apply secure HTTP headers.

Input Validation

All incoming requests are validated using Zod schemas.

CORS Protection

Only approved origins are permitted to access the API.

Password Security

Passwords are securely hashed using bcrypt.

JWT Authentication

Protected endpoints require valid JWT tokens.

Role-Based Access Control

Admin and Staff permissions are strictly separated.

Audit Logging

Critical activities are recorded for accountability.

Security Alerting

Suspicious activities generate security alerts.

4.2 Static Application Security Testing

Static security testing was performed using:

- GitHub CodeQL
- SonarQube

4.3 Dynamic Security Testing

OWASP ZAP was used to perform dynamic security testing.

4.4 Penetration Testing

Manual testing verified protection against:

- Unauthorized access
- Privilege escalation
- Input manipulation
- Authentication bypass attempts

4.5 Security Testing Results

Testing confirmed successful implementation of authentication, authorization, validation, logging, and monitoring controls.

Chapter 5: Container Security and Infrastructure Automation

5.1 Docker Security

The application was containerized using Docker while following container hardening practices.

- Security Measures
- Minimal attack surface
- Controlled dependencies
- Environment isolation
- Secure container configuration

5.2 Kubernetes Security

Kubernetes was used for orchestration and deployment.

Security controls include:

- Network Policies
- Secrets Management
- Security Contexts
- Resource Controls

5.3 Infrastructure as Code

Terraform was used to automate infrastructure provisioning.

5.4 Secrets Management

HashiCorp Vault was implemented for centralized secret storage.

Vault securely stores:

- Database credentials
- JWT secrets
- Application configuration secret

Chapter 6: Monitoring, DevSecOps, and AI Security

6.1 DevSecOps Pipeline

A DevSecOps pipeline was implemented using GitHub Actions.

Pipeline stages include:

- Source Code Validation
- Security Scanning
- Testing
- Build Verification
- Deployment Validation

6.2 Security Monitoring

Continuous monitoring provides visibility into:

- Login activity
- Inventory modifications
- Security events
- System anomalies

6.3 AI-Based Anomaly Detection

A lightweight anomaly detection mechanism was implemented to identify unusual system behavior and assist administrators in detecting suspicious activity.

Chapter 7: Results and Conclusion

7.1 Results

InventoryHub successfully achieved all planned objectives.

Major accomplishments include:

- Secure authentication and authorization
- Threat modelling and risk assessment
- Security-focused architecture
- Secure coding practices
- Static and dynamic testing
- Container and Kubernetes security
- Infrastructure automation using Terraform
- Secrets management using Vault
- DevSecOps integration
- Security monitoring and anomaly detection

7.2 Conclusion

InventoryHub demonstrates the practical application of secure software development principles throughout the software lifecycle. Security was integrated from requirements engineering to

deployment and monitoring. The implementation of threat modelling, secure architecture, testing, infrastructure automation, and DevSecOps practices resulted in a secure and maintainable inventory management platform. The project highlights the importance of embedding security into every stage of development to protect critical business systems against evolving cyber threats.

References

1. OWASP Foundation
2. NIST Cybersecurity Framework
3. Microsoft Threat Modelling Tool
4. Docker Documentation
5. Kubernetes Documentation
6. Terraform Documentation
7. HashiCorp Vault Documentation
8. GitHub CodeQL Documentation
9. SonarQube Documentation
10. OWASP ZAP Documentation